

NeuroGolf 2026

Implementation Plan — Research-Driven, 12-Day Sprint

Synthesizing 5 research streams into actionable steps

Executive Summary

This plan is built on five streams of fresh research: 4 detailed sub-agent reports totaling 2,359 lines (ARC taxonomy, ONNX minimization, LLM synthesis, codebase audit) plus direct extraction of four key Kaggle discussion threads from active competitors ranked 18th-528th. The research surfaced five findings that fundamentally reshape our strategy. The plan below converts those findings into 28 concrete implementation steps, organized into 6 phases over 12 days, with effort estimates, dependencies, and success criteria for each.

Five Research Findings That Drive the Plan

F1 — Scoring is fractional, not binary. Empirically verified by a 270th-ranked competitor: $\text{task_points} = (25 - \ln(\text{cost})) \times \text{held_out_fraction}$. A near-miss that gets 90% of test pairs correct earns 90% of the points. Strategy shift: implement the rule, not the examples.

F2 — MACs and node count are FREE; only params + intermediate tensors count. Confirmed by the host's May 4 update. The biggest single lever is memory golf: crop input to content bounding box, run the trace on the compact frame, pad back. One team dropped a net from 59.7M to 1.6M this way.

F3 — Top teams COMPILE reference ARC programs, not train CNNs. Compiled DSL rules generalize to held-out at 0.93-1.0 vs ~0.42 for trained memorizers. Hodel's arc-dsl (~160 primitives) ships `solve_{id}()` programs for many tasks. Transpiling those into opset-10 ONNX is the documented path to ~2× scores.

F4 — The grader uses ORT 1.24.4 with optimizations DISABLED. One mis-wired file (e.g., a TopK node the local runtime accepts) zeroes the entire submission.zip. We must build a faithful local replica and a per-file validator.

F5 — LLM synthesis at \$0.85/task gets 92.5% accuracy. Gemini 3.5 Flash High scores 92.5% on ARC-AGI public training. 400 tasks × \$0.85 = \$340 in API costs would theoretically solve ~370 tasks. Use onnxscript (not raw ONNX) as the LLM output format — it gives free parse-time validation.

1. Research Synthesis

Before drafting this plan, I launched five parallel research streams. Their outputs (saved as Markdown reports under /home/z/my-project/data/) total 2,359 lines of primary-source findings. The most actionable insights from each stream are summarized below; the full reports are referenced throughout.

1.1 Kaggle Discussion Forums (live competitor signals)

I fetched four key discussion threads (IDs 697079, 707993, 711989, 712047) and extracted the body text. Three findings stand out:

- "Compile, Don't Train" (souldrive, 270th, 21 days ago) — Posted a runnable Kaggle notebook proving that compiled opset-10 ONNX from ARC DSL programs beats trained CNNs by ~2x on held-out. Their honest ceiling: ~3,400 local points (below Bronze cutoff). The post lists 7 reusable ONNX-golf patterns.
- "NeuroGolf Survival Kit" (Georgy Mamarin, 528th, 11 days ago) — Lost a full submission to one bad TopK node. Built a public local scorer that reproduces the official ORT 1.24.4 cost computation exactly, plus a per-file validator + diagnose() function. TopK is NOT banned (myth busted).
- "Proposal: file-size scoring" (ymg_aq, 18th, 2 months ago) — Demonstrated a static-scorer exploit that can score 25 on arbitrary tasks (theoretical 10,000). Host pushed back. Implies the leaderboard may contain exploit-driven scores we can't easily replicate; focus on real algorithmic solutions instead.

1.2 ONNX Minimization (measured byte savings)

The ONNX minimization research measured every claim against our actual codebase. Concrete byte savings on our existing primitives:

Optimization	Before	After	Score Δ	Notes
Drop default Conv attrs (strides/dilations/group/pads=0)	653 B	602 B	+0.10	Pure attribute deletion
Drop kernel_shape (ORT infers from weight)	602 B	526 B	+0.13	Verified opset 11-21
int8 weights via QuantizeLinear→QLinearC onv→DequantizeLinear	526 B	392 B	+0.41	Argmax-identical to f32
Replace Constant nodes with initializers	—	−56 B each	—	Per Constant node
Strip producer_name + shorten graph name	—	−22 B	—	Per model
Shorten tensor names (conv_w → w)	—	−12 B/name	—	Per named tensor
Eliminate redundant Identity/Cast glue (argmax)	391 B	207 B	+0.62	Drop Cast + Identity

Headline: a valid identity network can be as small as 107 bytes (score 20.33). Our current identity is 130 bytes (score 20.13) — we're leaving 0.20 points per task on the floor just from metadata.

1.3 ARC-AGI Task Taxonomy (16 concept families)

The taxonomy research cross-walked Chollet's Core Knowledge priors, the ConceptARC 16-concept taxonomy, Hodel's arc-dsl (~160 primitives), and Icecuber's 142 unary functions against our local 400-task corpus. Each task family has a known minimum ONNX architecture:

Family	Tasks	Min Arch	Min Cost	Expected Score
Color substitution	~40	1×1 Conv	100	18.4
Geometric (flip/rot/transpose)	~30	Slice/Transpose	130	20.1
Scaling (k×)	~15	Resize	150	20.0
Tiling / Kronecker	~10	Tile + Mul	200	19.7
Cellular automaton	~50	3×3 Conv + bias	300	19.3
Flood fill / containment	~20	Unrolled max-propagation (H+W)	1500	17.7
Draw lines between markers	~15	Per-pixel conv + OR	2000	17.4
Object extraction (subsample)	~25	Permutation MatMul	500	18.9
Count → dimension	~15	ReduceSum + Reshape + Gemm	300	19.3
Symmetric completion (mirror+concat)	~20	Slice + Concat	180	19.9
Conditional transformation (if-then)	~30	Conv + Mul gate	500	18.9
Sorting / rearrangement	~10	Permutation MatMul	800	18.4
Pattern completion (fill blanks)	~20	Iterated dilation	1500	17.7
Other / irregular	~100	LLM-synthesized	varies	varies

The taxonomy identifies 16 distinct families covering ~290 of 400 tasks. The remaining ~100 irregular tasks require LLM synthesis. Realistic coverage ceiling: ~3400 local points (souldrive's published result).

1.4 LLM Program Synthesis (state-of-the-art)

The LLM synthesis research scraped the official ARC-AGI leaderboard. Headline numbers (cost per task, accuracy on ARC-AGI public training):

Model	Cost/Task	Accuracy	Notes
Gemini 3.1 Pro High	\$0.96	98.0%	Best accuracy; expensive
GPT-5.5 xHigh	\$1.87	95.0%	Strong but costly
Claude Opus 4.8 High	\$2.74	92.0%	Premium option
Gemini 3.5 Flash High	\$0.85	92.5%	Best price/performance
GPT-5.5 Medium	\$0.86	92.2%	Close second
GPT-4o (base, no CoT)	\$0.08	4.5%	Useless without CoT
Greenblatt GPT-4o + 8K samples	\$8.00	50.0%	2024 baseline; now obsolete

Key insight: a single Gemini 3.5 Flash High CoT call at \$0.85 now beats Greenblatt's 8,000-sample GPT-4o brute-force pipeline from 2024. The math has shifted — frontier CoT reasoning models do internally what Greenblatt brute-forced externally. Recommended pipeline: 32 samples + 1 revision round per task, validated against fresh ARC-GEN seeds. onnxscript (Microsoft) is the right output format — Python decorator that compiles to ONNX with parse-time validation.

1.5 Codebase Audit (bugs and gaps)

The audit found 5 confirmed bugs and 4 stub solvers that always return None. The most impactful findings:

- MirrorConcatSolver is broken — flips the 30×30 padded input instead of the actual grid, producing garbage in the padding region. Fix: bake expected output dims as Slice constants. Estimated +5 tasks.
- ScaleUpSolver emits malformed shape — output is (1,10,60,60) instead of (1,10,30,30). The local runtime falls back to lenient merge; the competition validator may not. Fix: add a final Slice. Protects 2 existing solves.
- filters.py is dead code — never wired into the dispatcher. 161 LOC of ConvFilter solver unused.
- multi_rule_ca is 14× too large — uses $10 \times 10 \times 3 \times 3 = 900$ params when typically only 4 colors are used. Pruning to $4 \times 4 \times 3 \times 3 = 36$ params drops cost from 4777 to ~1100, scoring +1.5 pts/task.
- No test suite, no regression detection, no golden results file — every change is a coin flip on whether it regresses.

2. Gap Analysis — Where We Are vs. Where We Need to Be

Combining the leaderboard data, the codebase audit, and the research findings, here is the precise gap we must close. The table is honest about what is achievable in 12 days vs. what would require more time.

2.1 Score Gap

Milestone	Score	Rank (est.)	Days Needed	Effort
Current (unsubmitted)	300	~2,750	0	—
Submit current + fix bugs (Day 1)	400	~2,650	1	Low
Cost optimization pass (Day 2-3)	550	~2,400	2	Medium
Extended CA + flood fill (Day 4-5)	1,500	~1,500	4	Medium
Object/extract/count solvers (Day 6-7)	2,500	~700	6	High
LLM synthesis loop (Day 8-10)	3,400	~300	9	High
Memory golf + byte optimization (Day 11)	3,800	~150	11	High
Final tuning + submit (Day 12)	4,000	~100	12	Medium
[Stretch] Multiple LLM revision rounds	5,000+	~50	+5 days	Very High
[Theoretical] Perfect compilation of all tasks	9,500	1	Not feasible	—

2.2 Coverage Gap (by task family)

Our current 16/400 solves vs. the taxonomy-derived ceiling for each family. The "gap" column shows how many more tasks each solver family could capture:

Family	Ceiling	Currently Solved	Gap	Solver Status
Color substitution	40	4	36	Exists, needs int8
Geometric	30	2	28	Exists, needs more transforms

Family	Ceiling	Currently Solved	Gap	Solver Status
Scaling	15	2	13	Exists, bug to fix
Tiling / Kronecker	10	1	9	Exists, only k=k case
Cellular automaton	50	3	47	Exists, needs multi-rule + 8-neighbor
Flood fill	20	0	20	MISSING — build unrolled max-propagation
Draw lines	15	0	15	MISSING — build Bresenham-equivalent
Object extraction	25	0	25	MISSING — build permutation MatMul
Count → dimension	15	0	15	MISSING — build ReduceSum + Reshape
Mirror concat	20	0	20	Exists but BROKEN — fix output dims
Conditional	30	0	30	MISSING — build Conv + Mul gate
Sorting	10	0	10	MISSING — build permutation MatMul
Pattern completion	20	0	20	MISSING — build iterated dilation
LLM-synthesized (long tail)	100	0	100	MISSING — build LLM pipeline
TOTAL	400	16	384	—

2.3 Cost Gap (per-task optimization)

Even with the same coverage, our per-task costs are 3-14× too high. The table shows the optimization potential for each existing solver:

Solver	Current Cost	Current Score	Optimized Cost	Optimized Score	Δ
identity	130	20.13	107	20.33	+0.20
color_map (1×1 conv)	753	18.38	200	19.70	+1.33
cellular_automaton	1427	17.74	300	19.30	+1.56
multi_rule_ca	4771	16.53	600	18.60	+2.07
kronecker	1123	17.98	600	18.60	+0.63
scale_up (bug-fixed)	341	19.17	200	19.70	+0.53
geom_transform	146	20.02	146	20.02	0.00
crop_top_left	130	20.13	130	20.13	0.00

Applied across our existing 16 solves, cost optimization alone adds ~6 points. Applied across 400 solves (assuming similar solver mix), it adds ~150 points.

3. Phase 0 — Foundation Fixes (Day 1, ~6 hours)

Before adding any new solvers, fix the confirmed bugs and build the missing infrastructure. These are blocking issues — every subsequent phase depends on them. All steps in this phase are independent and can be done in parallel.

Step 0.1 — Build faithful local scorer replica

Why: Per finding F4, the competition grader uses ORT 1.24.4 with optimizations disabled. Our current validator uses ORT 1.27 with default optimizations. We are validating against the wrong runtime. One bad file zeroes the entire submission.

What:

- Install onnxruntime==1.24.4 in a separate virtualenv (don't pollute our main env)
 - Build neurogolf/faithful_scorer.py that mirrors the official cost computation: params = sum of initializer element counts, memory = sum of intermediate tensor sizes (in bytes), cost = params + memory, score = max(1, 25 - ln(cost))
 - Add a diagnose(model) function that reports the exact gate a file dies at: load → size check → banned-op scan → runtime probe → scorability check
 - Add a pre-submit batch validator that runs all 400 files through the faithful scorer
- Effort: 2 hours · Dependency: none · Success: validator reproduces souldrive's published 6657-point local score on their notebook's output

Step 0.2 — Fix MirrorConcatSolver

Why: Audit found it builds the flip on the 30×30 padded input, producing garbage in the padding region. 5 tasks attempted, 0 solved.

What: Bake the expected output (H, W) as Slice constants so the concat happens on the cropped grid, not the padded one. Re-run on all 400 tasks to confirm solves.

Effort: 30 min · Dependency: Step 0.1 (for validation) · Success: +5 tasks solved (~+95 score)

Step 0.3 — Fix ScaleUpSolver shape bug

Why: Output is (1,10,60,60) instead of (1,10,30,30) when k>1. Local runtime falls back to lenient merge; competition grader may reject. Protects 2 existing solves.

What: Add a final Slice node to crop the (k×30, k×30) output back to (30, 30). Also relax the constraint that kh==kw to support asymmetric scaling (e.g., 2×3).

Effort: 30 min · Dependency: Step 0.1 · Success: 2 existing solves protected + 3-5 new tasks (~+57 score)

Step 0.4 — Wire filters.py into the dispatcher

Why: The ConvFilterSolver module (161 LOC) is dead code. Even though it's slow, it adds coverage.

What: Add filters.ConvFilterSolver() and filters.ColorSubstitutionSolver() to the dispatcher. Add a timeout guard (5s per task) so the slow ConvFilter doesn't dominate runtime.

Effort: 15 min · Dependency: none · Success: +0-3 tasks (~+0-50 score)

Step 0.5 — Remove stub solvers that always return None

Why: ColorCountSolver, FillBorderSolver, MaxColorSolver, BiasColorMapSolver all unconditionally return None. They waste dispatcher slots and obscure real signals.

What: Either implement them properly (see Phase 2) or remove them from the default solver list. Keep the stubs in the codebase for future implementation.

Effort: 15 min · Dependency: none · Success: dispatcher runs faster, cleaner signal

Step 0.6 — Set up regression testing

Why: No test suite means every change is a coin flip. We need a golden results file to diff against.

What:

- Create tests/test_regression.py that runs all 400 tasks through all solvers

- Save the per-task results as tests/golden_results.json
- Add a CI-like check: `python -m tests.test_regression --check` fails if any previously-solved task regresses
- Add per-solver unit tests for known-good outputs (e.g., `color_map` on a synthetic task)

Effort: 1.5 hours · Dependency: Step 0.1 · Success: regression detected automatically on every commit

Step 0.7 — Submit to Kaggle to validate I/O convention

Why: Per finding F4, we still haven't confirmed our I/O convention assumption. Even 300 points confirms we're not at zero.

What: Upload submission.zip via the Kaggle UI (user does this manually since we don't have API access). Document the score returned; if 0, immediately re-examine the I/O convention.

Effort: 5 min (user time) · Dependency: Steps 0.2-0.5 · Success: score > 0 on Kaggle

Phase 0 Exit Criteria

- Faithful local scorer matches ORT 1.24.4 behavior on a known-good submission
- MirrorConcat + ScaleUp solvers fixed and producing +5-8 new solves
- Regression test suite in place with golden results file
- Kaggle submission returns score > 0 (validates I/O convention)
- Estimated score after Phase 0: ~400 (rank ~2,650)

4. Phase 1 — Cost Optimization (Day 2, ~6 hours)

Per finding F2 (MACs are free; only params + intermediate tensors count) and the ONNX minimization research, every existing solver can be made smaller. This is the highest-ROI work per hour spent: each byte saved adds ln-improvement to every task that solver touches.

Step 1.1 — Strip metadata from every model

What: Edit `dsl.empty_model()` to:

```
model.ClearField('producer_name') model.ClearField('producer_version')
model.ClearField('doc_string') model.graph.name = 'g' # was 'neurogolf' (8 bytes saved)
```

Effort: 5 min · Gain: ~22 bytes per model = ~+0.04 score per task

Step 1.2 — Drop default Conv attributes

What: In `single_layer_conv2d` and `conv_stack`, remove `strides=[1,1]`, `dilations=[1,1]`, `group=1` when they're default. Remove `kernel_shape` entirely — ORT infers it from the weight tensor shape.

Effort: 20 min · Gain: ~95 bytes per Conv = ~+0.13 score per Conv-using task

Step 1.3 — Shorten tensor names

What: Rename all initializers and intermediates to single letters: `conv_w` → `w`, `conv_b` → `b`, `W_count` → `c`, etc. Tensor names appear in every node's inputs/outputs list, so the savings compound.

Effort: 30 min · Gain: ~12 bytes per named tensor

Step 1.4 — Add int8 quantization path for Conv weights

Why: The biggest single optimization. A 1×1 color_map conv has 100 float32 weights = 400 bytes. As int8, that's 100 bytes — a 4× saving. Verified argmax-identical to f32 in the ONNX minimization research.

What:

- Add `dsl.single_layer_conv2d_int8(weight, bias=None)` that emits `QuantizeLinear` → `QLinearConv` (no bias) → `DequantizeLinear`
- Add a float32 bias via a separate `Add` node if needed (`QLinearConv` with int32 bias is rejected by ORT 1.27 — known bug)
- Use this path automatically when the weight has > 30 elements (break-even point)
- Apply to `color_map`, `cellular_automaton`, `multi_rule_ca`, `conv_filter` solvers

Effort: 2 hours · Gain: ~260 bytes on `color_map` (753→392), ~1100 bytes on `multi_rule_ca` (4771→600) — adds ~+1.0 to +2.0 score per affected task

Step 1.5 — Replace Constant nodes with initializers

What: Every Constant node can be replaced by an initializer of the same name (initializers live in `graph.initializer` and are referenced by name like any other tensor). This saves ~56 bytes per Constant node.

Effort: 1 hour · Gain: ~56 bytes per Constant = ~+0.1 score per task with Constants

Step 1.6 — Add memory-golf pass to every solver

Why: Per finding F2, memory (intermediate tensor size) is the largest cost component, not params. Souldrive's post documents the technique: crop input to its content bounding box, run the trace on the compact frame, then pad back. One net dropped from 59.7M to 1.6M this way.

What:

- Build a utility `dsl.memory_golf_wrap(model, content_bbox)` that wraps any model with: `Pad-input-down` → `run-model` → `Pad-output-up`
- Detect content bounding box from the input grid (top-left, bottom-right non-zero cells)
- Apply this wrapper to every solver that produces intermediate tensors > 1000 elements
- Specifically target: `kronecker`, `multi_rule_ca`, `flood fill` (Phase 2)

Effort: 2 hours · Gain: potentially massive on large-intermediate solvers

Phase 1 Exit Criteria

- Every existing solver emits the smallest possible ONNX (verified byte count vs. Phase 0)
- int8 quantization active for all conv layers with > 30 weights
- Memory-golf wrapper available and applied where it helps
- Estimated score after Phase 1: ~550 (rank ~2,400) — same coverage, lower cost

5. Phase 2 — Missing Solver Families (Days 3-5, ~18 hours)

Per the gap analysis, 6 solver families are completely missing and would add ~125 tasks of coverage. Build them in priority order, largest gap first.

Step 2.1 — Extended CA solver (multi-rule + 8-neighbor)

Why: 47-task gap (largest single opportunity). Our current CA solver only handles single (X, Y, Z, threshold) rules with 4-neighbors. Many tasks need 8-neighbor rules or multiple rules combined.

What:

- Extend MultiRuleCASolver to try all three neighbor sets (4-neighbor, 8-neighbor, diagonal-only) — partially done, finish it
- Add CountColorCASolver: cell of color X with K neighbors of color Y becomes color (K mod 10) — handles count-to-color tasks
- Add PositionalCASolver: rules that fire only in specific rows/cols (e.g., "even rows only")
- Add ChainedCASolver: apply rule A then rule B in sequence (chain two CA models)

Effort: 4 hours · Gain: +30-50 tasks (~+550 score)

Step 2.2 — Flood fill / containment solver

Why: 20-task gap. ARC tasks like 00d62c1b ("fill enclosed region with color 4") require iterative neighbor propagation. Per souldrive's post, the opset-10 technique is unrolled max-propagation: seed each foreground cell with a unique id, propagate the neighbor-max for H+W steps (BFS flood unrolled to its worst-case diameter).

What: Build FloodFillSolver in solvers/flood_fill.py:

```
# Unrolled max-propagation ids = unique_id_per_cell * foreground for _ in range(H + W): # H+W
= 60 iterations max nbr = max(ids, shift_up(ids), shift_down(ids), shift_left(ids),
shift_right(ids)) ids = nbr * foreground # re-mask
```

Implementation note: Each iteration is a Max+Mul graph; H+W=60 iterations is large but each iteration is small. Apply memory-golf wrapper from Phase 1.6.

Effort: 3 hours · Gain: +8-15 tasks (~+150 score)

Step 2.3 — Object extraction (subsample) solver

Why: 25-task gap. Many ARC tasks present a grid containing multiple objects and ask you to extract one (e.g., the largest, or the one matching a marker). Souldrive's permutation-MatMul technique handles this without dynamic Slice.

What: Build ObjectExtractSolver:

- Detect objects via connected-component labeling (use the flood-fill primitive from 2.2)
- Compute a 0/1 keep-mask for the target object
- Build a permutation matrix R that scatters kept rows to top-left
- Apply MatMul(R, grid) for row compaction, MatMul(grid, C) for column compaction
- Output is a static 30×30 canvas with content packed top-left

Effort: 3 hours · Gain: +10-20 tasks (~+190 score)

Step 2.4 — Count → dimension solver

Why: 15-task gap. Tasks like "output a row of length N where N = count of color 5 in input". Requires ReduceSum + comparison + one-hot encoding.

What: Build CountDimSolver:

- Compute count = ReduceSum(input[channel=target_color])
- Build index vector [0, 1, ..., 29] as a Constant
- Compute Less(index, count) → (30,) bool
- Cast to float, one-hot encode to channel output_color

- Broadcast to (1, 10, 30, 30) via Expand
- Effort: 2 hours · Gain: +5-10 tasks (~+95 score)

Step 2.5 — Draw-lines-between-markers solver

Why: 15-task gap. Tasks like "draw a line of color C between the two markers of color M". Requires per-pixel conv that detects "on the line between two specific cells".

What: Build DrawLineSolver using a Bresenham-equivalent approach: for each pair of marker cells, the line between them can be expressed as a per-pixel gate = $(dx * (y - y1) == dy * (x - x1)) \& (between)$. Implement via element-wise Mul and comparison ops.

Effort: 3 hours · Gain: +5-10 tasks (~+95 score)

Step 2.6 — Conditional transformation solver

Why: 30-task gap (largest after CA). Tasks where a transformation applies only if some condition holds (e.g., "if cell is adjacent to color 5, replace with color 8, else keep").

What: Build ConditionalSolver that chains a CA-style rule with a color_map fallback: `out = where(rule_mask, Z, input)`. This is essentially the existing cellular_automaton solver generalized to arbitrary conditions.

Effort: 2 hours · Gain: +10-20 tasks (~+190 score)

Phase 2 Exit Criteria

- 6 new solver families implemented and tested
- All new solvers use int8 weights and memory-golf wrapper where applicable
- Estimated score after Phase 2: ~1,500 (rank ~1,500) — 80-100 tasks solved

6. Phase 3 — ARC DSL Transpiler (Days 6-8, ~18 hours)

Per finding F3, the documented winning strategy is to compile reference ARC DSL programs into opset-10 ONNX rather than training CNNs. Hodel's arc-dsl (github.com/hodel/arm) ships ~160 primitives and `solve_<id>()` programs for many of the 400 training tasks. The transpiler converts those Python programs into ONNX graphs.

Step 3.1 — Clone and audit Hodel's arc-dsl

What:

- Clone github.com/hodel/arm (or the equivalent arc-dsl repo)
- Catalog every primitive: input types, output types, ONNX-op mapping
- Catalog every `solve_<id>()` program: which task IDs have reference solutions
- Build a coverage matrix: 400 ARC tasks × 160 primitives → how many can we cover?

Effort: 2 hours · Output: `data/arc_dsl_audit.md`

Step 3.2 — Build ONNX transpiler core

What: Build `neurogolf/transpiler.py` that maps each arc-dsl primitive to an ONNX subgraph. Start with the most common primitives:

arc-dsl primitive	ONNX ops	Approx. cost
identity	Identity	107 B
color(c)	Constant (one-hot)	300 B
mask(color)	1×1 Conv + Mul	200 B
recolor({from: to})	1×1 Conv (int8)	200 B
flip_h / flip_v	Slice (reverse)	130 B
rotate_90 / rotate_180 / rotate_270	Transpose + Slice	180 B
crop(bbox)	Static Slice	130 B
paste(top_left, grid)	Pad + Add	200 B
dilate(mask)	3×3 MaxPool	300 B
erode(mask)	3×3 MinPool (via negation)	300 B
flood_fill(mask)	Unrolled max-propagation	1500 B
extract_objects(mask)	Connected-component labeling	1500 B
count(mask)	ReduceSum	150 B
compose(f, g)	Chain	f + g

Effort: 6 hours · Output: working transpiler for ~15 core primitives

Step 3.3 — Transpile reference solve_<id>() programs

What: For each task ID where arc-dsl has a reference solver, run the transpiler to produce an ONNX file. Validate locally; ship if correct.

Expected: ~50-100 tasks have reference solvers in arc-dsl that transpile cleanly. Another ~50-100 need manual transpiler fixes for primitives we didn't cover in Step 3.2.

Effort: 4 hours · Gain: +50-100 tasks (~+950 to +1900 score)

Step 3.4 — Generalize to tasks without reference solvers

What: For tasks without arc-dsl reference solvers, use the transpiler primitives manually. The Idea: synthesize a solver program by composing primitives based on the task signature (same as our current dispatcher, but using arc-dsl primitives which are more expressive).

Effort: 4 hours · Gain: +20-40 tasks (~+380 to +760 score)

Step 3.5 — Apply memory-golf and int8 optimization to all transpiled models

What: Re-run every transpiled model through the Phase 1 optimization passes. The transpiler naturally produces some intermediate tensors that can be golfed down.

Effort: 2 hours · Gain: +0.5-1.0 score per affected task

Phase 3 Exit Criteria

- ARC DSL transpiler operational with ~15 core primitives
- ~100-150 tasks solved via transpiled reference programs
- Estimated score after Phase 3: ~3,000 (rank ~400) — 150-200 tasks solved

7. Phase 4 — LLM Synthesis Loop (Days 9-10, ~10 hours)

Per finding F5, LLM synthesis at \$0.85/task (Gemini 3.5 Flash High) gets 92.5% accuracy on ARC-AGI public training. After Phases 0-3, we'll have ~150-200 tasks solved via DSL. Phase 4 picks up the remaining ~200-250 tasks that no DSL primitive can handle.

Step 4.1 — Install onnxscript and verify it works

Why: Per the LLM synthesis research, asking LLMs for raw ONNX protobuf fails constantly. onnxscript (Microsoft) provides a Python @script() decorator + op.* API that compiles to ONNX with free parse-time validation.

What:

```
pip install onnxscript # Verify: import onnxscript from onnxscript import op from
onnxscript.function import script @script() def color_map(input): return op.Conv(input, W) #
works!
```

Effort: 30 min · Output: working onnxscript environment

Step 4.2 — Build the LLM synthesis pipeline

What: Build neurogolf/llm_synth.py:

- For each unsolved task, construct a prompt with: (a) ARC task ID, (b) all input/output pairs as ASCII art, (c) the I/O tensor convention, (d) the ONNX op reference, (e) instructions to write onnxscript code that minimizes params+bytes
- Call Gemini 3.5 Flash High via the API (32 parallel samples per task)
- Execute each generated onnxscript snippet locally; validate against all pairs
- If at least one sample produces an eligible model, ship the smallest
- If none work, run one revision round: feed the LLM the error messages and ask for fixes

Prompt template (sketch):

You are competing in NeuroGolf 2026. Design the smallest possible ONNX network that transforms input to output for ARC task {TASK_ID}. I/O convention: - Input: (1, 10, 30, 30) float32 - one-hot color encoding, padded with 0 - Output: (1, 10, 30, 30) float32 - argmax over channels gives the answer Training pairs (5 shown): {ASCII_ART_OF_PAIRS} Write Python using onnxscript. Use only these ops: Identity, Slice, Concat, Transpose, Constant, Conv, Mul, Add, Sub, Pad, Resize, Tile, ArgMax, OneHot, Cast, ReduceSum, Gather, Greater, Less, Min, Max, Where. Minimize params + intermediate tensor bytes. Return only the Python code.

Effort: 4 hours · Output: working LLM synthesis pipeline

Step 4.3 — Run LLM synthesis on all remaining unsolved tasks

What: Iterate through the ~200-250 tasks that Phases 0-3 didn't solve. Budget: 32 samples × \$0.85/task × 250 tasks = ~\$6,800 in API costs. (If budget-constrained, prioritize by expected score gain: low-cost solvable tasks first.)

Note on cost: \$6,800 is more than we likely want to spend. Realistic budget: \$200-400 = ~25-50 tasks attempted with 32 samples each. Pick the 50 tasks where we have the most signal (e.g., tasks with similar structure to ones we already solved).

Effort: 4 hours (wallclock ~6-12 hours due to API latency) · Gain: +30-50 tasks (~+570 to +950 score)

Step 4.4 — Validate every LLM-synthesized model on fresh ARC-GEN seeds

Why: Per Georgy Mamarin's reply in discussion 711989: "Gate every lossy rewrite against fresh ARC-GEN generator seeds — the hidden set uses the same generator, different seeds. Catch 'passes locally, 0 on private' overfits pre-submit."

What:

- Install RE-ARC (procedural ARC task generator) or use the ARC-AGI-2 generator
- For each LLM-synthesized model, generate 5-10 fresh seeds of the same task family
- If the model fails any fresh seed, mark it as overfit and reject
- This step is critical: a model that scores 0 on private benchmark is worse than no model

Effort: 2 hours · Output: overfit-resistant LLM pipeline

Phase 4 Exit Criteria

- LLM synthesis pipeline operational with onnxscript output format
- Fresh-seed validation catches overfit models before submission
- Estimated score after Phase 4: ~3,800 (rank ~150) — 200-250 tasks solved

8. Phase 5 — Memory Golf & Final Push (Day 11, ~8 hours)

Per finding F2, memory (intermediate tensor bytes) is the largest cost component. Phase 5 applies aggressive memory-golf to every solver, then does a final byte-level optimization pass.

Step 5.1 — Apply memory-golf wrapper to every solver

What: For every solver, wrap its output with: Pad-input-down to content bbox → run-model → Pad-output-up. This means the model only operates on the actual content, not the 30×30 padded canvas. For a 5×5 input, this drops memory from 30×30=900 to 5×5=25 — a 36× reduction.

Effort: 3 hours · Gain: potentially large on tasks with small inputs but currently large intermediates

Step 5.2 — Run onnxsim.simplify on every model

What: Install onnxsim and run `onnxsim.simplify(model)` on every model. This performs constant folding, identity elimination, and graph simplification. Then re-run `ClearField('producer_name')` because onnxsim re-adds it.

Effort: 30 min · Gain: ~10-20% byte reduction on most models

Step 5.3 — Run scs4onnx for constant deduplication

What: `scs4onnx` (Simplify Constant-Shape) deduplicates Constants with identical values. If two solvers share the same Constant (e.g., a [1,1,1,1] tensor of ones), they get merged.

Effort: 30 min · Gain: ~5-10 bytes per duplicated Constant

Step 5.4 — Manual byte-level golfing on top-50 highest-cost models

What: Sort all 400 models by cost. For the top 50 (highest cost), manually inspect each ONNX graph and look for: redundant Identity nodes, unnecessary Casts, Constants that could be initializers, op simplifications (e.g., Conv with all-ones weight → Mul).

Effort: 3 hours · Gain: ~+0.5 score per affected model

Step 5.5 — Run final regression test and pre-submit validator

What: Run the full 400-task sweep through the faithful scorer (Step 0.1). Confirm no regressions vs. golden results. Run the pre-submit validator on every single ONNX file. Do not submit if any file fails validation — one bad file zeroes the entire submission.

Effort: 1 hour · Output: validated submission.zip ready to upload

Phase 5 Exit Criteria

- Every model has been through memory-golf, onnxsim, and scs4onnx
- Top 50 highest-cost models manually golfed
- Pre-submit validator passes on all 400 files
- Estimated score after Phase 5: ~4,200 (rank ~100)

9. Phase 6 — Final Submission & Monitoring (Day 12, ~4 hours)

Step 6.1 — Upload final submission.zip to Kaggle

What: User uploads submission.zip via Kaggle UI. Document the returned score; compare to local prediction. If significantly lower, identify which tasks scored 0 on private benchmark.

Effort: 15 min · Output: confirmed Kaggle score

Step 6.2 — Identify private-benchmark failures

What: If the returned score is significantly lower than our local prediction (~4200), some models passed public pairs but failed private. Use the per-task score breakdown (if Kaggle shows it) to identify which tasks scored 0. For each, examine whether the model was overfit (LLM-synthesized without fresh-seed validation) or genuinely wrong.

Effort: 1 hour · Output: list of failing tasks with causes

Step 6.3 — Final LLM synthesis round on identified weak tasks

What: For tasks that scored 0 on private, regenerate models with more samples (64 instead of 32) and stricter fresh-seed validation. Submit again if the daily submission limit allows.

Effort: 2 hours · Gain: recover 5-20 failing tasks

Step 6.4 — Document lessons learned

What: Update worklog.md with: final score, final rank, what worked, what didn't, lessons for next competition. This is valuable for any future ARC-AGI competitions.

Effort: 30 min · Output: comprehensive worklog

Phase 6 Exit Criteria

- Final submission uploaded and scored
- Any recoverable private-benchmark failures addressed
- Lessons documented in worklog
- Final estimated score: ~4,000-4,500 (rank ~80-120)

10. Risks, Dependencies, and Contingencies

10.1 Hard dependencies (must be available for plan to work)

Dependency	Why	Status	Contingency
Kaggle account (manual upload)	Submit submission.zip	Available (user)	N/A
Gemini 3.5 Flash High API access	LLM synthesis (Phase 4)	UNKNOWN — needs user API key	Use Claude Opus 4.8 (\$2.74/task) or skip Phase 4
onnxruntime 1.24.4 install	Faithful scorer (Phase 0)	Should install via pip	Use 1.27 with manual matching
onnxscript package	LLM output format (Phase 4)	pip install onnxscript	Fall back to manual ONNX generation
onnxsim, scs4onnx packages	Byte optimization (Phase 5)	pip install both	Manual constant folding
Hodel's arc-dsl repo	Reference programs (Phase 3)	Public on GitHub	Use only our own DSL
RE-ARC generator	Fresh-seed validation (Phase 4)	Public on GitHub	Skip overfit detection (risky)
\$200-400 API budget	LLM synthesis costs	User must provide	Reduce sample count or skip Phase 4

10.2 Schedule risks

Risk	Likelihood	Impact	Mitigation
Phase 4 (LLM) blocked by API access	Medium	High (-1000 score)	Skip Phase 4, extend Phase 3 (more transpilation)
Faithful scorer doesn't match grader	Low	Critical (submits may score 0)	Validate against souldrive's published notebook first
Hodel's arc-dsl doesn't transpile cleanly	Medium	High (-1500 score)	Build only the 15 core primitives ourselves
Memory-golf wrapper introduces bugs	Medium	Medium	Apply only to solvers where it's clearly beneficial
Time overruns push Phase 6 past deadline	Medium	Low	Phases are independently shippable; submit daily
Fresh-seed validation rejects too many models	Medium	Medium	Loosen validation: accept models that pass ≥ 4 of 5 fresh seeds

10.3 Decision points and pivots

At each phase exit, decide whether to continue to the next phase or pivot:

- After Phase 0 (Day 1): If Kaggle submission scores 0, the I/O convention is wrong — immediately halt all other work and re-derive the convention.
- After Phase 2 (Day 5): If coverage is < 60 tasks (vs. target 80-100), the CA/flood-fill solvers are failing — pivot to LLM synthesis early.
- After Phase 3 (Day 8): If transpiler covers < 100 tasks (vs. target 150), the arc-dsl approach isn't working — pivot fully to LLM synthesis.
- After Phase 4 (Day 10): If LLM synthesis succeeded on < 30 tasks, the API or prompts need work — extend Phase 4 by one day before Phase 5.
- At any point: If we cross rank ~ 100 on the leaderboard, the marginal value of further work drops sharply — consider stopping and locking in the rank.

11. Summary — One-Page View

The 12-Day Sprint at a Glance

Day	Phase	Work	Est. Score	Est. Rank
1	0 — Foundation	Faithful scorer, fix bugs, regression tests, submit	400	~2,650
2	1 — Cost Opt	int8, drop attrs, memory-golf wrapper	550	~2,400
3	2 — CA ext	Multi-rule CA, 8-neighbor	900	~2,100
4	2 — Flood/Extract	Flood fill, object extract	1,200	~1,800
5	2 — Count/Draw/Cond	Count→dim, draw line, conditional	1,500	~1,500
6	3 — DSL transpiler	Audit arc-dsl, build core	2,000	~1,100
7	3 — Transpile refs	Transpile solve_() programs	2,500	~700
8	3 — Generalize	Manual composition for unsolved	3,000	~400
9	4 — LLM setup	onnxscript, pipeline, prompts	3,200	~350
10	4 — LLM run	Synthesize 30-50 hard tasks	3,800	~150
11	5 — Memory golf	Apply to all, byte-level golf	4,200	~100
12	6 — Submit & monitor	Upload, debug, document	4,000-4,500	~80-120

Realistic Outcomes

Outcome	Probability	Score	Rank	Prize?
Pessimistic (multiple blockers)	20%	~2,000	~700	No medal
Realistic (most phases succeed)	50%	~3,500	~250	No medal
Optimistic (all phases hit targets)	25%	~4,500	~80	Bronze edge
Stretch (LLM exceeds expectations)	5%	~6,000+	~30	Silver possible

What This Plan Buys Us vs. Status Quo

- Status quo (no further work): ~300 score, rank ~2,750 — essentially last place.
- This plan, realistic outcome: ~3,500 score, rank ~250 — top 10% of leaderboard.
- This plan, optimistic outcome: ~4,500 score, rank ~80 — Bronze medal range.
- Theoretical max (not feasible): 9,500 score, rank 1 — would need 6 more months.

Bottom line: this plan converts a last-place submission into a credible top-10% finish, with a small (5-25%) chance at a medal. The single highest-leverage action is getting the LLM synthesis pipeline working — that alone could add 1,000+ points if we have API access. The single most critical infrastructure investment is the faithful local scorer — without it, we may submit zero-scoring files without knowing.

Immediate Next Action

Start Phase 0 immediately. The single most important step is Step 0.1 (build the faithful local scorer) because every subsequent step depends on accurate validation. Step 0.7 (submit to Kaggle) should happen on Day 1 to validate the I/O convention before we invest 11 more days of work on top of a wrong assumption.